Natural Language Processing

Ryan Levy

2026-03-24

I took CS 4770: Natural Language Processing in the Spring of my 4th year at the University of Virginia, taught by Professor Yangfeng Ji.

Given the worlds recent interest in LLMs, I thought I'd share my notes on the subject. I aim to present the information in a way that's accessible to the average reader, taking inspiration from Richard Feynman, American theoretical physicist, who notably explained complex concepts from start to finish in simple prose.

Introduction

Natural Language Processing (NLP) is the study of how machines attempt to understand language.

Text Classification

Imagine getting either of these messages from your boss:

> "Great work this week"
>
> "We need to talk about your performance"

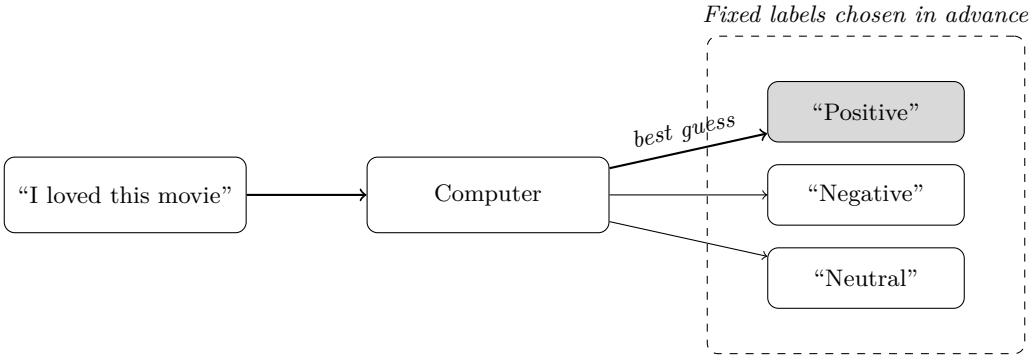
To you, it's obvious which would be classified as "positive" and "negative." To a computer, not so much.

This is where we'll start our discussion on NLP: how do computers classify text like we do?

Defining The Task

> Throughout this paper, we'll go over lots of examples of NLP. To keep us from getting confused, each time we set up a new example, we'll define the task in plain English. So that if you're confused later down the page, you can come back here to recenter.

Text classification is a when you give a computer some text (a word, a sentence, a paragraph) and it takes its best guess to match it to a list of pre-set labels.



Before we figure out how to get our computer to guess correctly, we need to convert our input text into a numeric form, since computers are designed to work with numbers much more efficiently than text.

Converting Text To Numbers

Instead of assigning our text to a single number, we assign it to a list of numbers, called a vector. Later on, this becomes useful as the computer can have a deeper understnaind of the text. One number might give us surface level knowledge of what the text means, but many numbers allows us to capture more of the complexities of language. It's like the difference between describing someone with one one dimension ("tall) vs three ("tall, dark hair, glasses"). More dimensions, better understanding.

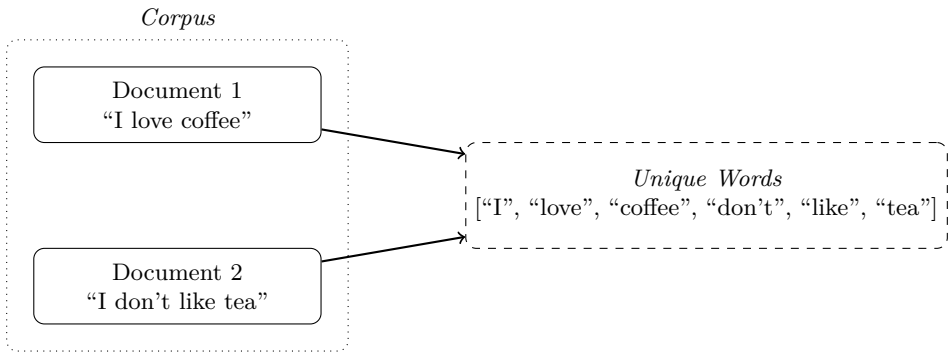
Bag of Words (BoW)

Bag of Words is the name of a method to convert text to a list of numbers.

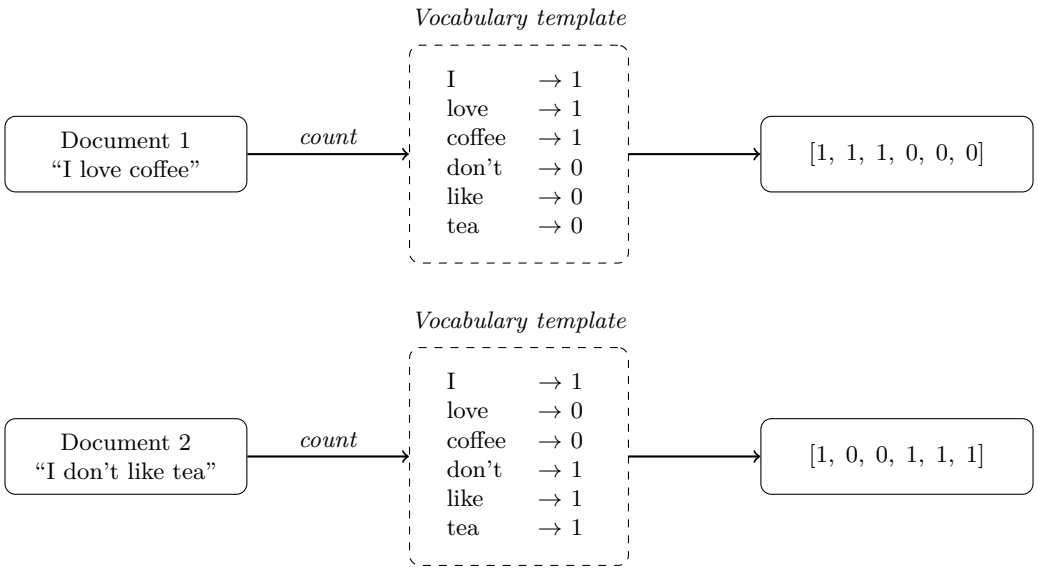
Let's say you have two documents. The first document says "I love coffee" and the second "I don't like tea."

We say that these documents make up our corpus, which is a collection of written texts. What Bag of Words

does, is it takes all the unique words[1] across all the documents in our corpus and puts them in a list.

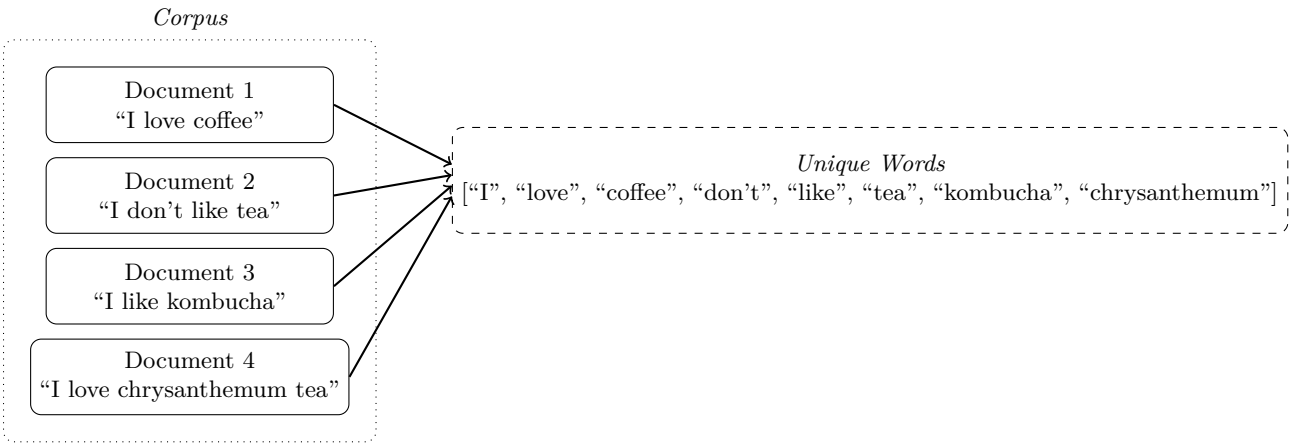


This unique list is called the vocabulary. BoW uses this vocabulary as a template to represent each document with the same structure. For each word in the vocabulary, it counts how many times that word appears in the document and records the result.



Notice the zero's in the spots where that document does not contain those vocabulary words. If one of the documents has words that are very rare, that will still be included in the vocabulary list.

[1] Technically it splits the text up into tokens, not words, but we'll get into that later.



	Doc 1	*Doc 2*	*Doc 3*	*Doc 4*
I	1	1	1	1
love	1	0	0	1
coffee	1	0	0	0
don't	0	1	0	0
like	0	1	1	0
tea	0	1	0	1
kombucha	0	0	1	0
chrysanthemum	0	0	0	1

With the introduction of just 2 rare words ("kombucha", "chrysanthemum"), our representations contain lots of 0's. Storing all these empty values create inefficiencies.

One way we combat this is removing rare (low-frequency) words in our corpus before creating the vocabulary.

Another issue is that with our current setup, words that are more frequent are treated as more important. Take the word "I." It appears in every document, but gives us little to no insight on the differences between them (i.e. whether one is "positive" or "negative").

To place more emphasis on words like "kombucha" and less on words like "I" and "the," we use a process called **TF-IDF**.

TF-IDF

We replace the raw count with the following:

$$c_{ij} = \mathrm{TF}_i \cdot \log \frac{N}{\mathrm{IDF}_i}$$

Where TF_i is the number of times word i appears in the document (the count we already had), N is the total number of documents in our corpus, and IDF_i is the number of documents that contain word i.

The key is the $\log \frac{N}{\mathrm{IDF}_i}$ part. If a word appears in every document, $\mathrm{IDF}_i = N$, so $\frac{N}{\mathrm{IDF}_i} = 1$, and $\log(1) = 0$. The entire value becomes zero. The word is effectively erased.

Let's see what this does to our table:

	Doc 1	*Doc 2*	*Doc 3*	*Doc 4*
I	0	0	0	0
love	0.30	0	0	0.30
coffee	0.60	0	0	0
don't	0	0.60	0	0
like	0	0.30	0.30	0
tea	0	0.30	0	0.30
kombucha	0	0	0.60	0
chrysanthemum	0	0	0	0.60

Notice how "I" — which appeared in every document — is now zero across the board. Meanwhile, words unique to a single document like "coffee" and "chrysanthemum" get the highest scores.

Another step you'll see is converting all the words to lowercase, so "Apple" and "apple" are treated the same[2].

Limitations of BoW

We now have a way to represent each document as a list of numbers. But let's see what happens when the computer tries to use these for classification.

Say we tell the computer that "I love coffee" is positive. Now it sees a new document: "I like tea." To us, these feel similar — both express enjoying a drink. But look at their vectors:

		I	*love*	*coffee*	*don't*	*like*	*tea*	*kombucha*	*chrysanth.*
"I love coffee"	→	0	0.30	0.60	0	0	0	0	0
"I like tea"	→	0	0	0	0	0.30	0.30	0	0

They're very different. Not a single value overlaps. As far as the computer can tell, these two documents have nothing in common.

Just by looking at the positions of the values in the vectors, we can visually see that they are different. But remember, a computer deals in numbers, so to assess whether the two are similar or not, the computer calculates a cosine similarity score.

Cosine Similarity

Cosine similarity compares two vectors and returns a score between 0 and 1. A score of 1 means the vectors are identical. A score of 0 means they have nothing in common.

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

The top part $(A \cdot B)$ multiplies each pair of matching values together and adds them up. If one vector has a number where the other has a zero, multiplying them produces a 0. So if no non-zero values overlap, the cosine similarity score is 0.

For example, if $A = [3, 0]$ and $B = [0, 5]$:

$$A \cdot B = (3 \times 0) + (0 \times 5) = 0$$

But if $A = [3, 2]$ and $B = [4, 2]$:

[2] The caveat here is if "Apple" is meant to represent Apple Inc. than there may be some issues. Trade offs like these exist over the entire methodology of NLP.

$$A \cdot B = (3 \times 4) + (2 \times 2) = 16$$

The more values that overlap, the higher the score.

The bottom part ($\|A\| \times \|B\|$) adjusts for the overall size of each vector. Without it, a vector with bigger numbers would always score higher, even if the pattern is the same. For example, $[2, 4]$ and $[4, 8]$ point in the same direction — one is just scaled up. The bottom part cancels that out, so we're comparing the pattern, not the size.

Let's compute the cosine similarity score for our two vectors:

	I	*love*	*coffee*	*don't*	*like*	*tea*	*kombucha*	*chrysanth.*	
A: "I love coffee"	0	0.30	0.60	0	0	0	0	0	
B: "I like tea"	0	0	0	0	0.30	0.30	0	0	
$A \times B$	0	+ 0	+ 0	+ 0	+ 0	+ 0	+ 0	+ 0	= **0**

Every pair multiplies to zero. The numerator is 0, so the entire score is **0**.

This score doesn't get fed into the classifier. The classifier still takes our vectors as input. But cosine similarity tells us whether those vectors are any good. If two sentences that mean similar things score a zero, our vectors aren't capturing meaning — and the classifier won't be able to either.

This is a fundamental limitation of Bag of Words: each word gets its own spot in the vector, and words that mean similar things have no connection to each other.

Better Representations

We need vectors where similar words have similar values in the same spots. Going back to our example: the computer learned that "I love coffee" is positive. Now it sees "I like tea" and needs to classify it. If "love" and "like" had similar values in the same spots, the vectors for these two sentences would overlap — and the computer would recognize "I like tea" as positive too.

Let's go back to our table. Up until now, we've been reading it by columns — each column is a document's vector. But what if we read it by rows instead? Each row tells us where a word appears across all our documents.

Look at the rows for "coffee" and "chrysanthemum." "Coffee" only appears in Doc 1. "Chrysanthemum" only appears in Doc 4. Their rows look nothing alike. But look at what else appears in Doc 1 and Doc 4 — "love." Both "coffee" and "chrysanthemum" show up in documents that also contain the word "love." They share context, even though they never appear in the same document.

Words that keep showing up around the same words probably mean similar things. That idea is called the **Distributional Hypothesis**.

> "You shall know a word by the company it keeps."
>
> — *(Firth, 1957)*

	"I love coffee"	*"I don't like tea"*	*"I like kombucha"*	*"I love chrysanthemum tea"*
	Doc 1	**Doc 2**	**Doc 3**	**Doc 4**
I	0	0	0	0
love	**0.30**	0	0	**0.30**
coffee	**0.60**	0	0	0
don't	0	0.60	0	0
like	0	0.30	0.30	0
tea	0	0.30	0	0.30
kombucha	0	0	0.60	0
chrysanthemum	0	0	0	**0.60**

different rows, but both appear in documents that contain "love"

However, the cosine similarity of the rows for "love" [0.30, 0, 0, 0.30] and "like" [0, 0.30, 0.30, 0] is:

$$A \cdot B = (0.30 \times 0) + (0 \times 0.30) + (0 \times 0.30) + (0.30 \times 0) = 0$$

We know these words are related, but the vectors don't reflect it. We need to adjust the numbers in this grid so that words with shared meaning end up with cosine similarity scores that are high.

Matrix Compression

We've been looking at our numbers as rows and columns. The table they form is called a **matrix**. Just like a list of numbers is called a vector, a 2D grid of numbers is called a matrix.

Our matrix is large and mostly zeros. Storing all that empty space is wasteful, and worse, the patterns we care about — like "love" and "like" being related — are hidden across different rows and columns. Compression lets us shrink the matrix down while keeping the important patterns intact.

Here's how compression works. Imagine you're rating 5 students on 6 tests: algebra, geometry, calculus, creative writing, poetry, and essays.

	Algebra	*Geometry*	*Calculus*	*Writing*	*Poetry*	*Essays*
Alice	90	85	92	40	35	45
Bob	30	35	25	88	90	85
Carol	88	80	85	42	38	40
Dan	35	40	30	82	85	90
Eve	50	55	48	50	52	55

That's 6 numbers per student. But look at the pattern: students who score high in algebra also score high in geometry and calculus. Students who score high in creative writing also score high in poetry and essays. Those 6 scores are really capturing 2 underlying traits: math ability and writing ability.

We can compress those 6 columns into 2:

	Alg	*Geo*	*Cal*	*Wri*	*Poe*	*Ess*
Alice	90	85	92	40	35	45
Bob	30	35	25	88	90	85
Carol	88	80	85	42	38	40
Dan	35	40	30	82	85	90
Eve	50	55	48	50	52	55

compress →

	Math	*Writing*
Alice	0.92	0.15
Bob	0.12	0.91
Carol	0.88	0.18
Dan	0.14	0.89
Eve	0.50	0.51

Each student now has just 2 numbers instead of 6. We lost some detail, but the important pattern is preserved: Alice and Carol are similar (both strong in math), Bob and Dan are similar (both strong in writing), and Eve is somewhere in the middle.

When we compress a matrix, we look for groups of scores that move together. Look at the algebra, geometry, and calculus columns side by side:

	Algebra	*Geometry*	*Calculus*
Alice	90	85	95
Bob	30	35	25
Carol	88	80	85
Dan	35	40	30
Eve	50	55	48

When algebra is high, geometry and calculus are high. When algebra is low, geometry and calculus are low. All three columns move together, capturing the same underlying trait: math ability.

Same with the writing side:

	Writing	*Poetry*	*Essays*
Alice	40	35	45
Bob	88	90	85
Carol	42	38	40
Dan	82	85	90
Eve	50	52	55

Writing, poetry, and essays move together. All three capture writing ability.

When groups of columns move together like this, you can replace them with a single number that summarizes the pattern. That's compression.

The math that finds these groups automatically is called **Singular Value Decomposition (SVD)**. It takes a matrix and breaks it into three smaller matrices:

$$M = U \times \Sigma \times V^T$$

V^T finds the groups — it figures out which columns move together and builds a recipe for combining them. Each row is a differnt recipe, and each recibe is a way to combine simmilar columns. For our students, it produces something like:

	Alg	*Geo*	*Cal*	*Wri*	*Poe*	*Ess*
Recipe 1	0.58	0.54	0.60	0.05	0.03	0.06
Recipe 2	0.04	0.06	0.03	0.55	0.57	0.56

Recipe 1 puts heavy weight on algebra, geometry, and calculus because those move together. Recipe 2 puts heavy weight on writing, poetry, and essays.

We can verify this works. To get Alice's Recipe 1 score, we multiply her original scores by the recipe weights and add them up:

$$(90 \times 0.58) + (85 \times 0.54) + (92 \times 0.60) + (40 \times 0.05) + (35 \times 0.03) + (45 \times 0.06)$$
$$= 52.2 + 45.9 + 55.2 + 2.0 + 1.1 + 2.7 = 159.1$$

That's a raw score. Σ scales it down. Σ is a matrix that ranks each recipe by how much **variation** it explains. Variation here means how spread out the scores are across students. A recipe that captures a lot of variation is picking up on a real, meaningful pattern — like math ability, where Alice scores 90 and Bob scores 30. A recipe that captures little variation is picking up on noise — tiny random differences that don't tell us much about the students.

	Recipe 1	*Recipe 2*
Recipe 1	**4.2**	0
Recipe 2	0	**3.1**

Recipe 1 (Math) has a higher value than Recipe 2 (Writing), meaning math ability explains more of the spread in scores. After scaling by Σ, Alice's raw score of 159.1 becomes 0.92.

The full result of the compression is U — each student's score on each recipe:

	Recipe 1 (Math)	*Recipe 2 (Writing)*
Alice	0.92	0.15
Bob	0.12	0.91
Carol	0.88	0.18
Dan	0.14	0.89
Eve	0.50	0.51

We choose how many recipes to keep. We kept 2. If we kept all 6, we'd get back the original matrix exactly — no compression. The recipes with the highest Σ values capture the strongest patterns. Everything else is noise we can safely throw away.

This is why compression leads to better representations. The strongest patterns — the ones that explain the most variation — are not always best split by documents, but by meaningful relationships revealed in patterns uncovered by the numbers. By keeping only those, we strip away the noise and keep the signal.

Applying Compression to Our Words

The same idea applies to our word-document matrix. We have 8 words and 4 documents. SVD compresses it down. If we keep 2 patterns, each word goes from a 4-number vector to a 2-number vector:

	Doc 1	*Doc 2*	*Doc 3*	*Doc 4*
I	0	0	0	0
love	0.30	0	0	0.30
coffee	0.60	0	0	0
don't	0	0.60	0	0
like	0	0.30	0.30	0
tea	0	0.30	0	0.30
kombucha	0	0	0.60	0
chrysanth.	0	0	0	0.60

$\xrightarrow{SVD}$

	Pattern 1	*Pattern 2*
I	0.01	0.01
love	0.52	0.08
coffee	0.44	−0.30
don't	−0.10	0.55
like	−0.08	0.50
tea	0.38	0.32
kombucha	−0.15	0.42
chrysanth.	0.40	−0.25

"love" and "like" had a cosine similarity of 0 before. Now look at their new vectors:

"love" $\rightarrow [0.52,\ 0.08]$

"like" $\rightarrow [-0.08,\ 0.50]$

These are no longer all zeros in different spots. They're short, dense vectors with values in every position.

Now look at "coffee" and "chrysanthemum" — the two words that shared "love" as context:

"coffee" $\rightarrow [0.44,\ -0.30]$

"chrysanthemum" $\rightarrow [0.40,\ -0.25]$

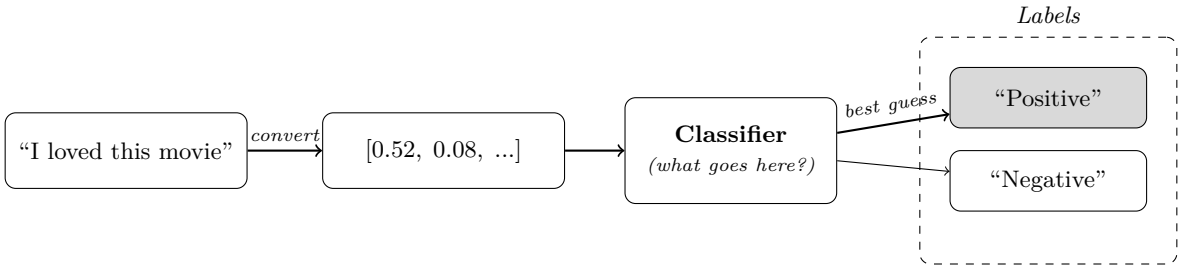
$$A \cdot B = (0.44 \times 0.40) + (-0.30 \times -0.25) = 0.176 + 0.075 = 0.251$$

Their cosine similarity is well above zero. The compression pulled out the hidden pattern we noticed earlier: both words appear in documents that contain "love." That relationship is now baked into the numbers.

This entire process — building a word-document matrix and compressing it with SVD — is called **Latent Semantic Analysis (LSA)**. "Latent" because the patterns were always there in the original matrix, hidden across the rows and columns. The compression brought them to the surface.

Classifying Our Number Representations of Text

Earlier, we showed classification as a black box — text goes in, a label comes out. Now we know how to turn text into numbers. The next step is to build what goes inside that box.



A classifier needs context to work with[3]. Say we have the following corpus:

	Document	*Label*
1	"I loved this movie"	Positive
2	"Great film, highly recommend"	Positive
3	"Terrible, waste of time"	Negative
4	"I hated every minute"	Negative
5	"What a wonderful experience"	Positive
6	"Awful acting, boring plot"	Negative
7	"Really enjoyed it"	?
8	"Not worth watching"	?

Each document has been converted into a vector using the methods we've covered. Documents 1–6 have labels attached. Documents 7 and 8 do not — those are the ones we want to classify.

This is similar to how humans learn. You weren't born knowing that "I loved this movie" is positive. You learned through experience — hearing people say similar things in similar situations, over and over, until you picked up the pattern. A classifier does the same thing, just with numbers.

We split our data into two groups. The first group — the **training data** — has the correct labels revealed.

[3] In some cases, classifiers can work without labeled context — this is called unsupervised learning. We'll get into that later.

The second group — the **test data** — does not. The classifier's job is to find patterns in the training data and use them to guess the labels for the test data.

The classifier does this through a **function**. A function takes an input, does something to it, and produces an output. A simple example:

$$f(x) = x \times w$$

Here, w is a **parameter** — a number we get to choose. If $w = 2$, then $f(3) = 6$. If $w = 10$, then $f(3) = 30$. Same input, different output, depending on the parameter.

Say we're trying to build a function where the correct answer for input 3 is 9. If $w = 2$, we get 6 — too low. If $w = 10$, we get 30 — too high. If $w = 3$, we get 9 — exactly right. Finding the right parameter is what makes the function work given our desired output.

For classification, the input is a vector (list of words in a document) and the output is a label. The function has parameters that we adjust until it correctly matches the training documents to their labels. If we find good parameters, the function should also work on the test data.

The Dummy Classifier

The simplest possible classifier only guesses the most likely label. If more data is labeled "positive" than "negative," it will guess every input is "positive."

Say our training data has 60 positive documents and 40 negative ones. The dummy classifier guesses "positive" every time. It would be right 60% of the time.

This is our baseline. If a real model can't beat 60%, we might as well not have built it.

The Simple Classifier

Let's take one step up from the dummy classifier. This time, we start with a list of words for each of our labels.

Positive words	**Negative words**
love	terrible
great	awful
wonderful	hated
enjoyed	boring
amazing	waste
recommend	disappointing

For each document, we count how many words from each list appear. Whichever count is higher becomes the label.

	Positive count	*Negative count*	*Label*
"I **loved** this **great** movie"	2	0	Positive
"**Terrible**, **boring** film"	0	2	Negative
"Really **enjoyed** it"	1	0	Positive
"It was okay"	0	0	?

This is better than guessing blindly, but it has clear limitations. We had to choose the word lists ourselves. Words not on either list get ignored entirely — "it was okay" gets no label at all. And every word on a list counts equally — "love" has the same weight as "recommend," even though one might be a stronger signal.

Logistic Regression

Instead of us picking the word lists by hand, we want the computer to figure out which words matter — and by how much — on its own.

Our function from earlier had one input (x) and one parameter (w):

$$f(x) = x \times w$$

Here, x is the input and w is the weight (w for weight, not word). Our vectors have many numbers — one for each word in the vocabulary.

If we have a weight for each word, and have two input words, the function looks like:

$$f(x_1, x_2) = x_1 \times w_1 + x_2 \times w_2$$

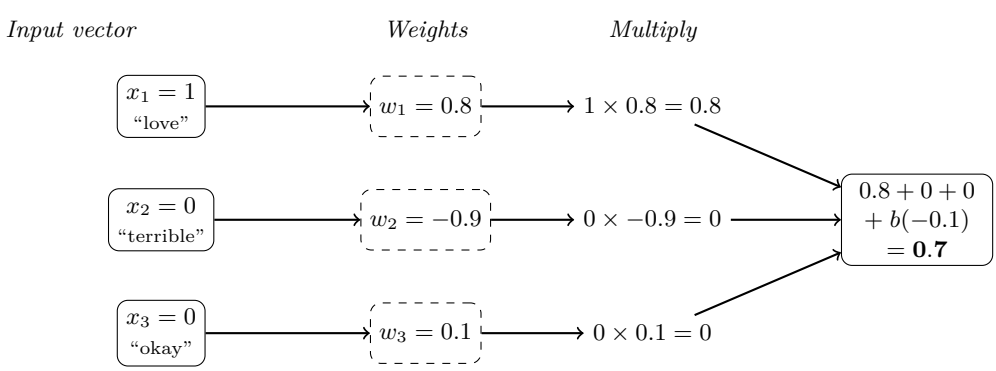
Each input gets multiplied by its own weight, and we add the results together. With three:

$$f(x_1, x_2, x_3) = x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3$$

The pattern continues for however many numbers are in our vector.

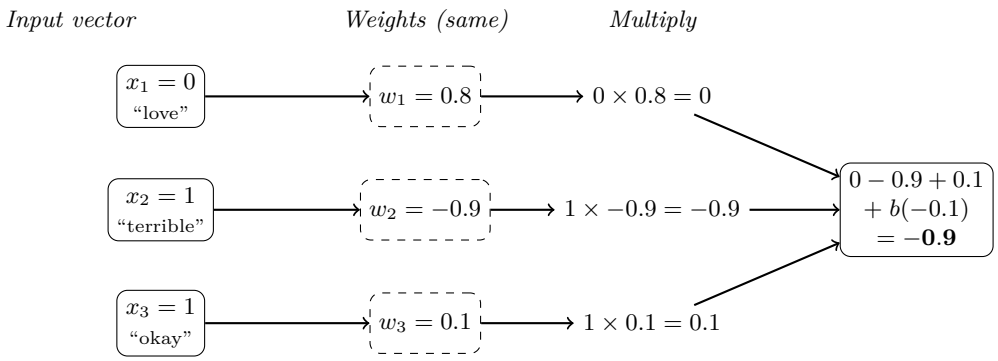
Let's make this concrete. Say we have a tiny vocabulary of just 3 words: "love," "terrible," and "okay." A document's vector has 3 numbers — one per word. The function also has 3 weights — one per word — plus an extra parameter b called the **bias**, which shifts the result up or down.

For now, let's assume we already have good weights. (We'll cover how to find them soon.) Here's the document "love" flowing through the function:



Notice what the weights are doing. "Love" has a high positive weight (0.8) — it pushes the result up. "Terrible" has a negative weight ($-$0.9) — it would push the result down if it appeared. "Okay" has a small weight (0.1) — it barely matters. Instead of us hand-picking word lists, the weights decide which words matter and by how much.

Here's the same weights applied to "terrible okay":



The first document scored 0.7. The second scored -0.9. Higher numbers lean positive, lower numbers lean negative. But these outputs have no fixed range — one document might score 0.7, another might score -300, another might score 42. We can't set a simple cutoff to decide positive vs negative when the range keeps changing.

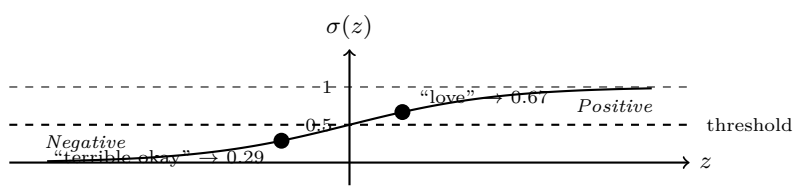
We need to put all outputs on the same scale. If every score fell between 0 and 1, we could pick a single cutoff — say 0.5 — and use it every time: above 0.5 is positive, below is negative.

The Sigmoid Function

The **sigmoid function** does exactly this. It takes any number and maps it to a value between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

A large positive number becomes close to 1. A large negative number becomes close to 0. Zero becomes exactly 0.5.



After the sigmoid, our two documents land on the curve. "Love" scored 0.7, which the sigmoid maps to 0.67 — above the 0.5 threshold, so it's labeled positive. "Terrible okay" scored -0.9, which maps to 0.29 — below the threshold, so it's negative.

Putting it all together, logistic regression takes a vector, multiplies it by the weights, adds the bias, and passes the result through the sigmoid:

$$h(x) = \sigma(w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b)$$

The output is always between 0 and 1. Above 0.5 means positive, below means negative. The only thing left is figuring out the right values for w and b.